บทที่ 12: การเขียนโปรแกรมแบบอะซิงโครนัสและเครือข่าย

12.1 บทนำ: โลกที่ไม่เคยรอ

ในบทที่ 11 เราได้จัดการสถานะที่ซับซ้อนด้วย Provider, Consumer และ MultiProvider จนแอปพลิเคชันมีโครงสร้างเป็นระเบียบและอัปเดตเฉพาะส่วนของหน้าจอที่จำเป็น อย่างไรก็ตาม แอปพลิเคชันเหล่านั้นยังคงทำงานอยู่ภายในขอบเขตของตัวเอง โดยใช้ข้อมูลที่เราสร้างขึ้นเอง (hard-coded data) แต่ในโลกแห่งความเป็นจริง แอปพลิเคชันที่ทรงพลังส่วนใหญ่จำเป็นต้องสื่อสารกับโลกภายนอก ไม่ว่าจะเป็นการดึงข้อมูลข่าวสารล่าสุดจากเซิร์ฟเวอร์, การบันทึกข้อมูลผู้ใช้ลงในฐานข้อมูลบนคลาวด์, หรือการอ่านไฟล์ขนาดใหญ่จากหน่วยความจำของอุปกรณ์

ภาพที่ช่วยให้เห็นข้อจำกัดนี้ชัดที่สุดคือห้องทำงานห้องหนึ่งที่ตกแต่งไว้อย่างดี ข้าวของทุกชิ้นวางเป็นระเบียบ แสงไฟส่องสว่างพอดี เป็นพื้นที่ที่ทำงานได้อย่างไร้ที่ติ แต่ห้องนั้นไม่มีหน้าต่างและไม่มีประตู มันถูกปิดตายอย่างสมบูรณ์ ข้อมูลทุกอย่างในห้องคือสิ่งที่ถูกใส่เตรียมไว้ล่วงหน้าเท่านั้น ไม่มีข่าวสารใดจากโลกภายนอกเข้ามาได้เลย

นั่นคือสภาพของแอปพลิเคชันที่เราสร้างจนจบบทที่ 11 โครงสร้างภายในเป็นระเบียบและจัดการสถานะได้ดีเยี่ยม แต่ยังอยู่อย่างโดดเดี่ยวบนข้อมูลที่ฝังไว้ในโค้ด เปิดแอปขึ้นมากี่ครั้งก็เห็นสิ่งเดิมซ้ำแล้วซ้ำเล่า งานของบทนี้คือการเจาะหน้าต่างและติดประตูให้ห้องนั้น เพื่อพาแอปพลิเคชันออกไปสื่อสารกับโลกภายนอกที่ชื่อว่าอินเทอร์เน็ต

บทที่ 11 ปิดท้ายไว้ด้วยคำถามที่ยังไม่ได้ตอบ นั่นคือ ระหว่างที่ข้อมูลยังเดินทางมาจากเซิร์ฟเวอร์ไม่ถึง provider ควรบอกหน้าจอว่าอย่างไร ถ้าตอบว่ารายการว่างเปล่า ผู้ใช้จะเข้าใจผิดว่าไม่มีสินค้าเลย และถ้าการเชื่อมต่อล้มเหลว ใครควรเป็นผู้จำข้อผิดพลาดนั้นไว้เพื่อแสดงปุ่มลองใหม่ บทนี้คือคำตอบของคำถามนั้น โดยเริ่มจากเครื่องมือระดับหน้าจอเดียวคือ FutureBuilder แล้วจึงยกคำตอบชุดเดียวกันขึ้นไปไว้ใน ChangeNotifier ในหัวข้อ 12.8

การดำเนินการเหล่านี้มีลักษณะร่วมกันอย่างหนึ่งคือ **ต้องใช้เวลา** และเราไม่สามารถคาดเดาได้แน่ชัดว่าจะใช้เวลานานเท่าใด หากแอปพลิเคชันของเรารอคอยการทำงานเหล่านี้จนเสร็จสิ้นโดยไม่ทำอะไรเลย จะส่งผลให้หน้าจอค้าง (freeze) และสร้างประสบการณ์ที่เลวร้ายให้กับผู้ใช้

บทนี้ต่อยอดจากพื้นฐานการเขียนโปรแกรมแบบอะซิงโครนัสในภาษา Dart ที่ได้ศึกษาไปแล้วใน บทที่ 2.2 โดยมุ่งไปที่การนำความรู้นั้นมาใช้จริงบน Flutter ร่วมกับงานเครือข่าย หากคุณยังไม่มั่นใจในเรื่อง Future, async, await หรือ try-catch ควรทบทวนบทที่ 2.2 ก่อน เนื้อหาที่จะได้เรียนรู้ในบทนี้ประกอบด้วย

- **อะซิงโครนัสในบริบทของ Flutter:** ข้อจำกัดของ UI isolate และงบประมาณเวลาต่อหนึ่งเฟรม ซึ่งทำให้งานเครือข่ายต้องเป็นอะซิงโครนัสเสมอ พร้อมทบทวนเครื่องมือจากบทที่ 2.2 อย่างย่อ
- **ข้อควรระวังเมื่อใช้ async ร่วมกับวิดเจ็ต:** ตำแหน่งที่ถูกต้องในการเริ่มงานอะซิงโครนัส การตรวจสอบ mounted และการใช้ BuildContext ข้าม await
- **FutureBuilder:** วิดเจ็ตของ Flutter ที่ออกแบบมาโดยเฉพาะเพื่อสร้าง UI ที่ตอบสนองต่อผลลัพธ์จากการทำงานแบบอะซิงโครนัสได้อย่างสง่างาม
- **การดึงข้อมูลจากอินเทอร์เน็ต (Networking):** นำความรู้ทั้งหมดมาประยุกต์ใช้เพื่อดึงข้อมูลจริงจาก Public API โดยใช้แพ็กเกจ http
- **การแปลงข้อมูล JSON:** เรียนรู้วิธีแปลงข้อมูลในรูปแบบ JSON ที่ได้รับจาก API ให้กลายเป็นอ็อบเจกต์ Dart ที่สามารถนำไปใช้งานได้
- **การเชื่อมข้อมูลจากเครือข่ายเข้ากับ Provider:** ต่อยอดการจัดการสถานะจากบทที่ 11 ให้รองรับข้อมูลที่มาแบบอะซิงโครนัส ซึ่งเป็นรูปแบบเดียวกับที่จะใช้กับฐานข้อมูลในบทที่ 13
- การรับมือกับความล้มเหลว: งานเครือข่ายล้มเหลวได้หลายแบบ ทั้งต่อไม่ติด ตอบช้าเกินรอ และตอบกลับมาไม่ใช่อย่างที่คาด เราจะแยกแยะแต่ละแบบ กำหนด timeout และแปลงข้อผิดพลาดให้เป็นข้อความที่ผู้ใช้อ่านรู้เรื่องพร้อมปุ่มลองใหม่
- การแปลงข้อมูลก้อนใหญ่โดยไม่ทำให้หน้าจอกระตุก: เมื่อข้อมูลที่ได้รับมีขนาดใหญ่จนการแปลง JSON กินเวลาเกินงบประมาณต่อหนึ่งเฟรม เราจะย้ายงานนั้นไปทำใน isolate อื่นด้วย compute

เมื่อจบบทนี้ คุณจะสามารถสร้างแอปพลิเคชันที่สามารถทำงานเบื้องหลังที่ซับซ้อน เช่น การดึงข้อมูลจากเครือข่าย โดยที่ส่วนติดต่อผู้ใช้ยังคงตอบสนองต่อผู้ใช้ได้อย่างลื่นไหล

12.2 จากภาษา Dart สู่ Flutter: สิ่งที่เปลี่ยนไปเมื่อ async มาอยู่ในวิดเจ็ต

หลักการของการทำงานแบบอะซิงโครนัสในภาษา Dart ได้อธิบายไว้อย่างละเอียดแล้วใน บทที่ 2.2 การเขียนโปรแกรมแบบอะซิงโครนัสในภาษา Dart ตั้งแต่กลไก Event Loop, อ็อบเจกต์ Future, การรับผลลัพธ์ด้วย .then() และ .catchError(), คำสงวน async และ await, การดักจับข้อผิดพลาดด้วย try-catch ไปจนถึง Stream และ StreamController บทนี้จึงไม่อธิบายเนื้อหาดังกล่าวซ้ำอีก หัวข้อนี้จะทบทวนเพียงสั้น ๆ ว่าต้องรู้อะไรมาก่อน แล้วจึงเข้าสู่ประเด็นใหม่ที่บทที่ 2.2 ไม่ได้ครอบคลุม นั่นคือสิ่งที่เปลี่ยนไปเมื่อนำงานอะซิงโครนัสมาไว้ในวิดเจ็ตของ Flutter จริง

ก่อนเริ่มบทนี้ ขอให้ตรวจสอบว่าคุณยังจำสาระสำคัญต่อไปนี้ได้ หากข้อใดยังไม่ชัดเจน ขอให้กลับไปทบทวนหัวข้อที่ระบุไว้ในวงเล็บก่อน

- **Future (บทที่ 2.2.2):** อ็อบเจกต์ที่แทน "ค่าเดียวที่จะได้รับในอนาคต" มีได้สามสถานะ คือ ยังไม่เสร็จสิ้น เสร็จสิ้นพร้อมค่า และเสร็จสิ้นพร้อมข้อผิดพลาด
- **async และ await (บทที่ 2.2.3):** คำสงวนที่ทำให้โค้ดอะซิงโครนัสอ่านเรียงบรรทัดได้เหมือนโค้ดซิงโครนัส โดย await หยุดรออยู่เฉพาะภายในฟังก์ชันนั้น ไม่ได้หยุดการทำงานของทั้งโปรแกรม [1]
- **try-catch (บทที่ 2.2.4):** กลไกดักจับข้อผิดพลาดของงานอะซิงโครนัส ซึ่งจะถูกใช้อย่างหนักในบทนี้ เพราะงานเครือข่ายล้มเหลวได้เสมอ
- **Stream (บทที่ 2.2.5 และ 2.2.6):** ลำดับข้อมูลหลายค่าที่ทยอยเข้ามาตามเวลา ต่างจาก Future ที่ให้ผลลัพธ์เพียงครั้งเดียว บทนี้เน้นที่ Future เป็นหลัก เนื่องจากการเรียก API แบบ request-response หนึ่งครั้งให้ผลลัพธ์เพียงชุดเดียว

เหตุใดการบล็อกจึงร้ายแรงเป็นพิเศษบน Flutter

บทที่ 2.2 อธิบายผลของการบล็อกไว้ในระดับของภาษา แต่เมื่อมาอยู่บน Flutter ผลกระทบจะรุนแรงและผู้ใช้มองเห็นได้ทันที เพราะโค้ด Dart ของแอปเกือบทั้งหมด ทั้งตรรกะทางธุรกิจและการสร้างวิดเจ็ต ทำงานอยู่บน isolate เดียวกันที่เรียกว่า UI isolate ซึ่งเป็นตัวเดียวกับที่รับผิดชอบการวาดหน้าจอในแต่ละเฟรม [2]

เพื่อให้ภาพเคลื่อนไหวลื่นไหลที่ 60 เฟรมต่อวินาที เฟรมเวิร์กมีเวลาราว 16 มิลลิวินาทีต่อหนึ่งเฟรมเท่านั้นในการคำนวณและวาดหน้าจอ และบนอุปกรณ์รุ่นใหม่ที่จอรีเฟรช 90 หรือ 120 เฮิรตซ์ งบประมาณนี้จะเหลือเพียงราว 11 และ 8 มิลลิวินาทีตามลำดับ [3] หากโค้ดของเรายึดครอง UI isolate ไว้นานกว่านั้น เฟรมจะถูกข้ามไปและเกิดอาการกระตุก (jank) ที่สังเกตเห็นได้ และหากยึดครองไว้นานหลายวินาที เช่น การรอผลลัพธ์จากเซิร์ฟเวอร์ ระบบปฏิบัติการอาจรายงานว่าแอปไม่ตอบสนอง

การเรียก API หนึ่งครั้งใช้เวลาตั้งแต่หลักร้อยมิลลิวินาทีไปจนถึงหลายวินาที ซึ่งมากกว่างบประมาณ 16 มิลลิวินาทีอยู่หลายเท่า ด้วยเหตุนี้ งานเครือข่ายทุกชนิดจึงต้องเป็นอะซิงโครนัสเสมอโดยไม่มีข้อยกเว้น

อุปมา: ร้านอาหารที่มีพนักงานเสิร์ฟเพียงคนเดียว

งบประมาณ 16 มิลลิวินาทีต่อหนึ่งเฟรมนั้นสั้นกว่าที่คนทั่วไปรู้สึก การกระพริบตาหนึ่งครั้งใช้เวลาราวหนึ่งร้อยถึงสี่ร้อยมิลลิวินาที นั่นคือช้ากว่างบประมาณต่อเฟรมหลายเท่า ในขณะที่การเรียก API หนึ่งครั้งกินเวลาตั้งแต่ร้อยมิลลิวินาทีไปจนถึงหลายวินาที

บทที่ 11 เคยเปรียบระบบจัดการสถานะกับร้านอาหาร โดยให้ provider เป็นพ่อครัวและ Consumer เป็นพนักงานเสิร์ฟ บทนี้ขยายภาพเดิมด้วยข้อเท็จจริงเพิ่มอีกข้อหนึ่ง คือร้านนี้มีพนักงานเสิร์ฟอยู่เพียงคนเดียว พนักงานคนนั้นคือ UI isolate

ถ้าพนักงานเสิร์ฟรับออเดอร์จากโต๊ะหนึ่ง เดินไปสั่งในครัว แล้วเลือกยืนรออยู่หน้าเตาจนอาหารจานนั้นเสร็จ นั่นคือการทำงานแบบซิงโครนัส ผลที่ตามมาไม่ใช่แค่โต๊ะนั้นได้อาหารช้า แต่ลูกค้าทุกโต๊ะที่เหลือในร้านไม่มีใครมารับออเดอร์ ไม่มีใครเติมน้ำ ร้านทั้งร้านหยุดชะงักพร้อมกัน เพราะมีพนักงานอยู่คนเดียว

วิธีที่ถูกคือจดออเดอร์ส่งเข้าครัวแล้วเดินไปรับลูกค้าโต๊ะอื่นต่อทันที เมื่ออาหารเสร็จพ่อครัวกดกริ่งเรียก พนักงานจึงกลับมารับไปเสิร์ฟ นั่นคือการทำงานแบบอะซิงโครนัส งานเครือข่ายทุกชนิดต้องทำแบบนี้เท่านั้น เพราะเสิร์ฟคนเดียวคนนี้มีหน้าที่วาดหน้าจอทุกเฟรมด้วย

ข้อควรระวังเมื่อใช้ async ร่วมกับวิดเจ็ต

มีข้อผิดพลาดสามประการที่พบบ่อยเมื่อนำ Future มาใช้กับวิดเจ็ต ซึ่งไม่ปรากฏในบทที่ 2.2 เพราะเป็นประเด็นที่เกิดจากวงจรชีวิตของวิดเจ็ตโดยเฉพาะ

- **อย่าเริ่มงานอะซิงโครนัสในเมธอด build:** เมธอด build อาจถูกเรียกซ้ำหลายครั้งต่อวินาที หากเรียกฟังก์ชันดึงข้อมูลไว้ในนั้น จะเกิดการยิงคำขอไปยังเซิร์ฟเวอร์ซ้ำไม่สิ้นสุด ให้เรียกใน initState แล้วเก็บ Future ไว้ในตัวแปรของคลาส State แทน
- **ตรวจสอบ mounted ก่อนเรียก setState หลัง await:** ระหว่างที่รอผลลัพธ์ ผู้ใช้อาจกดย้อนกลับออกจากหน้าจอไปแล้ว ทำให้วิดเจ็ตถูกถอดออกจากต้นไม้ วิดเจ็ต การเรียก setState ในจังหวะนั้นจะทำให้เกิดข้อผิดพลาด จึงควรเขียน if (!mounted) return; คั่นไว้ก่อนเสมอ ทั้งนี้ mounted เป็น getter ที่มีอยู่ในคลาส State เท่านั้น ไม่มีใน StatelessWidget และไม่มีในคลาส ChangeNotifier ที่จะใช้ในหัวข้อ 12.8 ซึ่งมีวิธีรับมือคนละแบบ
- **ระวังการใช้ BuildContext ข้าม await:** ตัวแปร context ที่ถือไว้ก่อน await อาจใช้การไม่ได้อีกเมื่อการรอสิ้นสุดลง หากจำเป็นต้องใช้ Navigator หรือ ScaffoldMessenger หลัง await ให้ดึงอ็อบเจกต์ที่ต้องการเก็บไว้ในตัวแปรก่อนเริ่มรอ หรือตรวจสอบ mounted ก่อนนำ context ไปใช้

ผลของการเรียกใน build เห็นได้ชัดเมื่อคิดถึงปริมาณ เมธอด build ไม่ได้ถูกเรียกครั้งเดียวตอนเปิดหน้าจอ มันถูกเรียกใหม่ทุกครั้งที่มีภาพเคลื่อนไหวขยับ ทุกครั้งที่แป้นพิมพ์เด้งขึ้นมา และทุกครั้งที่สถานะบางอย่างในวิดเจ็ตแม่เปลี่ยน จึงถูกเรียกซ้ำได้หลายสิบครั้งในหนึ่งวินาที ถ้าคำสั่งดึงข้อมูลอยู่ในนั้น แอปของเราจะสาดคำขอไปที่เซิร์ฟเวอร์รัว ๆ ทุกครั้งที่ผู้ใช้เลื่อนหน้าจอ ผลที่ได้คือแอปช้าลง ปริมาณข้อมูลที่ใช้พุ่งสูง และในกรณีร้ายแรงคือเซิร์ฟเวอร์ล่มเพราะถูกโจมตีจากแอปของเราเอง

ส่วนเหตุผลของการตรวจ mounted เทียบได้กับการคุยโทรศัพท์ สมมติว่าเราคุยกับเพื่อนอยู่ แล้วบอกเพื่อนว่าถือสายรอสักครู่ จากนั้นวางหูไว้แล้ววิ่งไปทำธุระ เมื่อกลับมาพร้อมคำตอบเราก็พูดใส่หูโทรศัพท์รวดเดียวจบ โดยไม่ได้ตรวจก่อนเลยว่าเพื่อนคนนั้นรอนานจนวางสายไปตั้งแต่ห้านาทีที่แล้ว โค้ดที่อยู่หลัง await ทำงานแบบนั้นทุกครั้ง มันจะกลับมาทำงานเสมอเมื่อข้อมูลมาถึง แต่ผู้ใช้ไม่ได้อดทนรอตลอดเวลา เขาอาจกดย้อนกลับออกจากหน้าจอไปแล้ว วิดเจ็ตถูกถอดออกจากต้นไม้และทำลายทิ้ง ขณะที่ฟังก์ชันดึงข้อมูลยังทำงานอยู่เบื้องหลัง การเรียก setState ในจังหวะนั้นคือการพยายามอัปเดตหน้าจอที่ไม่มีอยู่จริงอีกแล้ว

ตัวอย่างโครงสร้างที่ถูกต้องเป็นดังนี้

```dart
class _NewsScreenState extends State<NewsScreen> {
  // ถูกต้อง: เก็บ Future ไว้ในตัวแปรของ State
  late Future<String> _headline;

  @override
  void initState() {
    super.initState();
    _headline = fetchHeadline(); // เริ่มงานครั้งเดียวตอนสร้างหน้าจอ
  }

  // การสั่งโหลดใหม่: สร้าง Future ตัวใหม่ แล้วให้ FutureBuilder เริ่มรอใหม่
  // กรณีนี้ไม่มี await คั่น จึงไม่ต้องตรวจสอบ mounted
  void _reload() {
    setState(() {
      _headline = fetchHeadline();
    });
  }
```

```
  @override
  Widget build(BuildContext context) {
    // ห้ามเรียก fetchHeadline() ตรงนี้ เพราะ build ถูกเรียกซ้ำได้หลายครั้ง
    // และจะได้ Future ตัวใหม่ทุกครั้ง ทำให้วงล้อหมุนไม่รู้จบ
    return FutureBuilder<String>(
      future: _headline,
      builder: (context, snapshot) {
        // สร้างวิดเจ็ตตามสถานะของ snapshot (ดูรายละเอียดในหัวข้อ 12.3)
        return Text(snapshot.data ?? 'กำลังโหลด...');
      },
    );
  }
}
```

ส่วนข้อควรระวังข้อที่สามเรื่อง BuildContext ข้าม await นั้น มีรูปแบบการเขียนที่ปลอดภัยสองแบบให้เลือกใช้ตามสถานการณ์ แบบแรกเหมาะเมื่อรู้ล่วงหน้าว่าจะต้องใช้อะไรบ้าง แบบที่สองเหมาะเมื่อโค้ดหลัง await มีเงื่อนไขซับซ้อนกว่านั้น

```
// ทางเลือกที่ 1: ดึงอ็อบเจกต์ที่ต้องใช้เก็บไว้ในตัวแปรก่อนเริ่มรอ
Future<void> _saveAndClose() async {
  final messenger = ScaffoldMessenger.of(context); // ดึงก่อน await
  final navigator = Navigator.of(context);

  await saveToServer();

  // ไม่ต้องแตะ context อีกแล้ว จึงปลอดภัยแม้หน้าจอจะถูกปิดไปก่อน
  messenger.showSnackBar(const SnackBar(content: Text('บันทึกแล้ว')));
  navigator.pop();
}

// ทางเลือกที่ 2: ตรวจสอบ mounted ก่อน แล้วจึงใช้ context หลัง await
Future<void> _saveAndNotify() async {
  await saveToServer();

  if (!mounted) return; // หน้าจอถูกปิดไปแล้ว ไม่ต้องทำอะไรต่อ
  ScaffoldMessenger.of(context).showSnackBar(
    const SnackBar(content: Text('บันทึกแล้ว')),
  );
}
```

12.3 การแสดงผลข้อมูลจาก Future ใน UI ด้วย FutureBuilder

เรารู้วิธีจัดการกับ Future ในโค้ด Dart มาแล้วจากบทที่ 2.2 แต่จะนำผลลัพธ์ไปแสดงใน UI ของ Flutter ได้อย่างไร? เราคงไม่อยากเขียนตรรกะการจัดการสถานะ (กำลังโหลด, เสร็จสิ้น, ผิดพลาด) ด้วย setState เองทั้งหมด เพราะจะทำให้โค้ดซับซ้อนและเกิดข้อผิดพลาดได้ง่าย

Flutter มีวิดเจ็ตที่ยอดเยี่ยมสำหรับงานนี้โดยเฉพาะ นั่นคือ **FutureBuilder**

FutureBuilder เป็น StatefulWidget ที่สร้างตัวเองขึ้นมาใหม่ตามสถานะล่าสุดของ Future ที่เราส่งให้ มันช่วยจัดการวงจรชีวิตของ Future ให้เราโดยอัตโนมัติ [4]

หากเปรียบเทียบกับบทที่ 11 จะเห็นความคล้ายคลึงกันอย่างชัดเจน Consumer วาดส่วนของ UI ใหม่เมื่อ ChangeNotifier แจ้งว่าสถานะเปลี่ยน ส่วน FutureBuilder วาดส่วนของ UI ใหม่เมื่อสถานะของ Future เปลี่ยน ต่างกันตรงที่ Future ให้ผลลัพธ์เพียงครั้งเดียวแล้วจบ ในขณะที่ ChangeNotifier แจ้งเตือนได้ไม่จำกัดจำนวนครั้ง

คุณสมบัติที่สำคัญ:

วิธีที่ตรงที่สุดในการอธิบายว่า FutureBuilder ช่วยอะไร คือลองนับงานที่หัวข้อ 12.2 บอกให้เราทำด้วยมือ ทั้งเรียกฟังก์ชันใน initState เก็บ Future ไว้ในตัวแปรของ State ตรวจ mounted ก่อน setState สร้างตัวแปร boolean ไว้จำว่ากำลังโหลดอยู่หรือไม่ และสร้างอีกตัวไว้จำข้อผิดพลาด งานทั้งหมดนี้อยู่ใต้ฝากระโปรงของ FutureBuilder เรียบร้อยแล้ว หน้าที่ที่เหลือของเรามีเพียงการเขียนฟังก์ชัน builder เพื่อบอกว่าในแต่ละสถานะอยากให้หน้าตาของแอปเป็นแบบไหน โดยรับข้อมูลสถานะผ่านพารามิเตอร์ที่ชื่อ snapshot

- **future:** รับอ็อบเจกต์ Future ที่เราต้องการจะ "ฟัง"
- **builder:** เป็นฟังก์ชันที่รับ (BuildContext context, AsyncSnapshot snapshot) และต้องคืนค่าวิดเจ็ต builder นี้จะถูกเรียกหลายครั้งเมื่อสถานะของ Future เปลี่ยนไป

AsyncSnapshot คืออะไร?

snapshot คืออ็อบเจกต์ที่เก็บข้อมูลเกี่ยวกับสถานะล่าสุดของ Future ณ เวลาที่ builder ถูกเรียก มี property ที่สำคัญคือ:

- **connectionState:** บอกสถานะการเชื่อมต่อของ Future (เช่น ConnectionState.waiting, ConnectionState.done)
- **hasData:** เป็น true ถ้า Future เสร็จสิ้นพร้อมข้อมูล
- **data:** คือข้อมูลที่ Future ส่งคืนมา (ใช้ได้เมื่อ hasData เป็น true)
- **hasError:** เป็น true ถ้า Future เสร็จสิ้นพร้อมข้อผิดพลาด
- **error:** คืออ็อบเจกต์ข้อผิดพลาด (ใช้ได้เมื่อ hasError เป็น true)

รูปแบบการใช้งานทั่วไป:

```
FutureBuilder<String>( // ระบุชนิดข้อมูลของ Future
  future: myFutureString, // Future ที่เรากำลังรอ
  builder: (context, snapshot) {
    // 1. ตรวจสอบสถานะการเชื่อมต่อ
    if (snapshot.connectionState == ConnectionState.waiting) {
      // ขณะที่ Future กำลังทำงาน ให้แสดงวงล้อหมุน
      return const CircularProgressIndicator();
    }
    // 2. เมื่อ Future ทำงานเสร็จแล้ว ให้ตรวจสอบข้อผิดพลาด
    else if (snapshot.hasError) {
      // หากเกิดข้อผิดพลาด ให้แสดงข้อความ
      return Text('เกิดข้อผิดพลาด: ${snapshot.error}');
    }
    // 3. หากไม่มีข้อผิดพลาดและมีข้อมูล
```

```
    else if (snapshot.hasData) {
      // นำข้อมูล (snapshot.data) มาแสดงผล
      return Text('ข้อมูลที่ได้รับ: ${snapshot.data}');
    }
    // 4. สถานะอื่นๆ (เช่น ยังไม่เริ่ม)
    else {
      return const Text('ยังไม่มีข้อมูล');
    }
  },
)
```

สถานะที่แท้จริงของ AsyncSnapshot และลำดับการตรวจที่ถูกต้อง

ตัวอย่างข้างต้นเขียนตามลำดับที่พบได้ทั่วไปในบทเรียนออนไลน์ คือตรวจ ConnectionState.waiting ก่อน แล้วจึงตรวจข้อผิดพลาดและข้อมูล ลำดับนี้ใช้ได้ดีเมื่อหน้าจอโหลดข้อมูลเพียงครั้งเดียว แต่จะให้ผลที่ไม่พึงประสงค์ทันทีที่เราเพิ่มความสามารถสั่งโหลดซ้ำเข้าไป ก่อนจะไปถึงจุดนั้น ต้องเข้าใจก่อนว่า connectionState มีค่าที่เป็นไปได้สี่ค่า ไม่ใช่สองค่า

- **ConnectionState.none** ยังไม่มี Future ให้รับฟัง เกิดขึ้นเมื่อพารามิเตอร์ future เป็น null หรือในจังหวะที่เราสลับไปใช้ Future ตัวใหม่ ฟังก์ชัน builder ถูกเรียกด้วยสถานะนี้ได้เช่นกัน จึงต้องมีทางออกเสมอ
- **ConnectionState.waiting** มี Future แล้วและกำลังรอผลลัพธ์
- **ConnectionState.active** ใช้กับข้อมูลที่ทยอยเข้ามาเป็นชุด ซึ่งเป็นเรื่องของ StreamBuilder แทบไม่พบในการใช้งานกับ Future
- **ConnectionState.done** Future ทำงานจบแล้ว แต่คำว่าจบยังไม่ได้บอกว่าสำเร็จหรือล้มเหลว ต้องตรวจ hasError และ hasData ต่อเสมอ

ประเด็นที่ต้องระวังคือ เมื่อเราเปลี่ยนค่าพารามิเตอร์ future ให้เป็น Future ตัวใหม่เพื่อสั่งโหลดซ้ำ FutureBuilder จะยกเลิกการรับฟังตัวเก่าแล้วรับฟังตัวใหม่ แต่ยังคง**ข้อมูลของรอบก่อน**ไว้ใน snapshot ผลคือในจังหวะนั้น connectionState เป็น waiting พร้อมกับที่ hasData ยังเป็น true อยู่

ถ้าโค้ดของเราตรวจ waiting เป็นเงื่อนไขแรกและคืนวงล้อหมุนทันที รายการที่ผู้ใช้กำลังอ่านอยู่จะหายไปทั้งหน้าแล้วกลับมาใหม่ เกิดอาการกระพริบทุกครั้งที่รีเฟรช ทางที่ดีกว่าคือตรวจข้อผิดพลาดก่อน แล้วตรวจว่ามีข้อมูลหรือไม่ และคืนวงล้อหมุนเฉพาะกรณีที่ยังไม่มีข้อมูลใด ๆ เลย ดังตัวอย่างในหัวข้อ 12.6

นอกจากนี้ FutureBuilder ยังรับพารามิเตอร์ initialData เพื่อกำหนดค่าเริ่มต้นให้ snapshot ได้ ซึ่งมีประโยชน์เมื่อเรามีข้อมูลจากที่อื่นอยู่ในมือแล้ว เช่นข้อมูลที่เคยบันทึกไว้ในเครื่อง และต้องการให้ผู้ใช้เห็นทันทีตั้งแต่เฟรมแรกระหว่างที่ข้อมูลชุดใหม่ยังเดินทางมาไม่ถึง

12.4 ปฏิบัติการย่อย: สร้าง UI ที่ตอบสนองต่อ Future

เราจะใช้ FutureBuilder แสดงผลลัพธ์จากฟังก์ชันที่จำลองงานซึ่งใช้เวลานานด้วย Future.delayed ตามที่ได้ศึกษาไว้ในบทที่ 2.2.2 โดยตัวฟังก์ชันรวมอยู่ในคลาส State ที่จะสร้างในขั้นตอนถัดไปแล้ว

คำถามที่ควรถามก่อนลงมือคือ ทำไมต้องเสียเวลาจำลองการรอคอยแบบปลอม ๆ ด้วย Future.delayed ในเมื่อเราต่ออินเทอร์เน็ตจริงไปเลยก็ได้ คำตอบคือการแยกปัญหาออกเป็นส่วน ๆ เป็นทักษะการไล่หาข้อผิดพลาดที่สำคัญที่สุดอย่างหนึ่ง เมื่อจำลองการรอคอยเอง เราควบคุมได้ว่าจะให้สำเร็จหรือให้ล้มเหลว และควบคุมได้ว่าจะรอกี่วินาที เราจึงออกแบบและทดสอบ UI ของทั้งสามสถานะให้ครบก่อนได้ ว่าวงล้อหมุนแสดงผลถูกต้องหรือไม่ ครบสามวินาทีแล้วข้อมูลสลับขึ้นมาแทนได้เนียน

หรือเปล่า

ประโยชน์ข้อสำคัญอยู่ตอนที่เกิดปัญหา ถ้าเราต่อเครือข่ายจริงไปพร้อมกับเขียน UI ใหม่ แล้วหน้าจอไม่แสดงอะไรเลย เราจะแยกไม่ออกว่าต้นเหตุอยู่ที่เครือข่าย ที่ข้อมูลที่ได้กลับมา หรือที่โค้ด UI ของเราเอง เมื่อโครงสร้าง UI ผ่านการทดสอบด้วยข้อมูลจำลองแล้ว การสลับไปดึงข้อมูลจริงในหัวข้อ 12.5 จึงเหลือตัวแปรที่อาจผิดพลาดเพียงด้านเดียว

1. **คำแนะนำ:** สร้างโปรเจกต์ Flutter ใหม่ หรือใช้หน้าว่างๆ ในโปรเจกต์ปัจจุบัน
2. **คำแนะนำ:** สร้าง StatefulWidget ขึ้นมา ในคลาส State ให้ประกาศ Future แต่ **อย่า** เรียกฟังก์ชันใน build โดยตรง เพราะจะทำให้ฟังก์ชันถูกเรียกทุกครั้งที่ build ทำงาน ให้เรียกใน initState แทน

```dart
class FutureDemoScreen extends StatefulWidget {
  const FutureDemoScreen({super.key});
  @override
  State<FutureDemoScreen> createState() => _FutureDemoScreenState();
}

class _FutureDemoScreenState extends State<FutureDemoScreen> {
  // ประกาศ Future เพื่อเก็บผลลัพธ์
  late Future<String> _calculation;

  Future<String> simulateSlowOperation() {
    return Future.delayed(
      const Duration(seconds: 3),
      () {
        // ลองยกเลิกคอมเมนต์สามบรรทัดนี้เพื่อทดสอบกรณีเกิดข้อผิดพลาด
        // if (true) {
        //   throw Exception('เกิดข้อผิดพลาดโดยตั้งใจ');
        // }
        return 'ข้อมูลจากการทำงานที่ใช้เวลานาน';
      },
    );
  }

  @override
  void initState() {
    super.initState();
    // เรียกฟังก์ชันและเก็บ Future ไว้ในตัวแปรใน initState
    _calculation = simulateSlowOperation();
  }

  // เมธอด build ของขั้นตอนถัดไปจะมาแทรกตรงนี้
  // นั่นคือ "ภายใน" คลาสนี้ ก่อนวงเล็บปีกกาปิดคลาสที่อยู่ด้านล่าง
}
```

3. **คำแนะนำ:** เพิ่มเมธอด build ต่อไปนี้เข้าไปในคลาส _FutureDemoScreenState โดยแทรกไว้ตรงตำแหน่งที่คอมเมนต์ระบุไว้ในโค้ดข้างต้น ไม่ใช่วางต่อท้ายหลังวงเล็บปีกกาปิดคลาส

```
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('FutureBuilder Demo')),
    body: Center(
      child: FutureBuilder<String>(
        future: _calculation, // ส่ง Future ของเราเข้าไป
        builder: (context, snapshot) {
          if (snapshot.connectionState == ConnectionState.waiting) {
            return const Column(
              mainAxisAlignment: MainAxisAlignment.center,
              children: [
                CircularProgressIndicator(),
                SizedBox(height: 10),
                Text('กำลังโหลดข้อมูล...'),
              ],
            );
          } else if (snapshot.hasError) {
            return Text('เกิดข้อผิดพลาด: ${snapshot.error}', style: const
TextStyle(color: Colors.red));
          } else if (snapshot.hasData) {
            return Text(
              'สำเร็จ! ได้รับข้อมูล: ${snapshot.data}',
              style: const TextStyle(fontSize: 18, color: Colors.green),
            );
          } else {
            return const Text('ไม่มีข้อมูล');
          }
        },
      ),
    ),
  );
}
```

คำแนะนำ: ตั้งค่า FutureDemoScreen ให้เป็นหน้าแรกของแอป โดยกำหนด home: const FutureDemoScreen() ใน MaterialApp ภายในไฟล์ main.dart มิฉะนั้นเมื่อรันแอปจะยังเห็นหน้าจอตั้งต้นของโปรเจกต์เหมือนเดิม

4. **คำแนะนำ:** รันแอปพลิเคชัน

   ผลลัพธ์ที่คาดหวัง:

   - เมื่อแอปเริ่ม คุณจะเห็นวงล้อหมุนพร้อมข้อความ "กำลังโหลดข้อมูล..." เป็นเวลา 3 วินาที
   - หลังจาก 3 วินาที UI จะอัปเดตและแสดงข้อความ "สำเร็จ! ได้รับข้อมูล: ข้อมูลจากการทำงานที่ใช้เวลานาน"

- (ทางเลือก) ลองยกเลิกคอมเมนต์ throw Exception(...) ในฟังก์ชันจำลอง แล้วรันใหม่ คุณจะเห็นข้อความแสดงข้อผิดพลาดสีแดงแทน

12.5 การสื่อสารกับโลกภายนอก: การดึงข้อมูลจากอินเทอร์เน็ต

ตอนนี้เราพร้อมที่จะนำความรู้ทั้งหมดมาประยุกต์ใช้กับสถานการณ์จริง: การดึงข้อมูลจาก Public API [5] เราจะใช้ JSONPlaceholder ซึ่งเป็น API ฟรีสำหรับทดสอบและสร้างต้นแบบ

เราจะดึงรายการ "todos" จาก https://jsonplaceholder.typicode.com/todos

ขั้นตอนที่ 1: การเพิ่ม http Package

ในการสร้าง HTTP request เราจำเป็นต้องใช้แพ็กเกจ http [6]

1. **คำแนะนำ:** เปิดไฟล์ pubspec.yaml
2. **คำแนะนำ:** เพิ่ม http ใน dependencies

```
dependencies:
  flutter:
    sdk: flutter
  http: ^1.6.0 # ตรวจสอบเวอร์ชันล่าสุดบน pub.dev
```

3. **คำแนะนำ:** รัน flutter pub get ใน terminal

สิ่งที่ต้องทำเพิ่มก่อนนำไปสร้างฉบับ release: เพียงเพิ่มแพ็กเกจยังไม่พอ แอปต้องได้รับอนุญาตจากระบบปฏิบัติการให้ออกอินเทอร์เน็ตด้วย บน Android โครงร่างโปรเจกต์ของ Flutter ประกาศสิทธิ์ INTERNET ไว้ให้เฉพาะในไฟล์ manifest ของโหมด debug และ profile เท่านั้น ระหว่างพัฒนาจึงไม่พบปัญหา แต่เมื่อสั่ง flutter build apk เพื่อสร้างฉบับ release แอปจะเชื่อมต่อไม่ได้เลยและมักแสดงเพียงข้อความว่าเชื่อมต่อล้มเหลว ซึ่งหาสาเหตุยากมากเพราะโค้ดฝั่ง Dart ไม่มีอะไรผิด วิธีแก้คือประกาศสิทธิ์นี้ในไฟล์ manifest ของ main ด้วย [7]

```
<!-- android/app/src/main/AndroidManifest.xml -->
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
    <uses-permission android:name="android.permission.INTERNET"/>
    <application android:label="api_demo" ...>
```

หากพัฒนาบน macOS ต้องเปิดสิทธิ์ของแอปพลิเคชันฝั่งลูกข่ายเพิ่มเติมในไฟล์ macos/Runner/DebugProfile.entitlements และ macos/Runner/Release.entitlements โดยตั้งคีย์ com.apple.security.network.client ให้เป็นจริง ส่วน iOS และ Android ในโหมด debug ใช้งานได้ทันทีโดยไม่ต้องตั้งค่าใด ๆ

ขั้นตอนที่ 2: การสร้าง Data Model สำหรับข้อมูล

ข้อมูลที่ได้จาก API จะอยู่ในรูปแบบ JSON (JavaScript Object Notation) ซึ่งเป็นรูปแบบข้อความที่คอมพิวเตอร์อ่านง่าย เพื่อให้ทำงานกับข้อมูลนี้ใน Dart ได้อย่างปลอดภัยและสะดวก เราควรสร้าง **Model Class** เพื่อแปลง JSON ให้เป็นอ็อบเจกต์ Dart [8]

คำถามที่ผู้เรียนมักถามตรงนี้คือ ทำไมเซิร์ฟเวอร์ไม่ส่งอ็อบเจกต์ของภาษา Dart กลับมาให้ตรง ๆ จะได้ไม่ต้องแปลง คำตอบคือเซิร์ฟเวอร์ไม่ได้เขียนด้วยภาษา Dart มันอาจเขียนด้วย Python, Node.js หรือ Go การจะคุยกันข้ามภาษาได้จึงต้องมีภาษากลางที่ทุกฝ่ายเข้าใจตรงกัน JSON ทำหน้าที่นั้น มันเป็นเพียงข้อความที่เบาและอ่านง่าย ไม่ผูกกับภาษาใดภาษาหนึ่ง

ผลที่ตามมาคือสิ่งที่ Dart ได้รับกลับมาเป็นแค่ก้อนข้อความยาว ๆ ที่เต็มไปด้วยวงเล็บและเครื่องหมายคำพูด ภาษา Dart ซึ่งเคร่งครัดเรื่องชนิดข้อมูลมองก้อนนั้นไม่ต่างจากภาษาต่างดาว มันไม่รู้ว่าข้างในมีกี่ฟิลด์ ชนิดอะไร หรือซ้อนกันลึกแค่ไหน ถ้าเราหยิบค่าจาก Map ดิบ ๆ ด้วยชื่อคีย์ที่เป็นข้อความไปใช้ในโค้ด UI โดยตรง พิมพ์ชื่อคีย์ผิดเพียงตัวอักษรเดียวแอปก็พังตอนรันโดยที่ตัววิเคราะห์โค้ดไม่เคยเตือนเลย

Model Class ที่มี factory constructor ชื่อ fromJson จึงทำหน้าที่เหมือนล่ามแปลภาษา มันรับข้อความต่างดาวเข้ามา ตรวจความถูกต้อง แล้วแปลงให้เป็นอ็อบเจกต์ Dart ที่แอปไว้ใจได้ ส่งต่อไปมาได้ และถ้าเราพิมพ์ชื่อฟิลด์ผิด เราจะรู้ตั้งแต่ตอนเขียนโค้ด ไม่ใช่ตอนผู้ใช้กดเปิดหน้าจอ กระบวนการนี้ทำงานเป็นสองจังหวะ คือ jsonDecode แปลงข้อความให้เป็นโครงสร้าง Map ก่อน แล้ว fromJson จึงเข้าไปหยิบข้อมูลทีละช่องมาใส่ตัวแปรที่ระบุชนิดไว้ชัดเจน

ข้อมูล Todo แต่ละรายการมีลักษณะดังนี้:

```
{
  "userId": 1,
  "id": 1,
  "title": "delectus aut autem",
  "completed": false
}
```

1. **คำแนะนำ:** สร้างไฟล์ใหม่ lib/models/todo.dart
2. **คำแนะนำ:** สร้างคลาส Todo พร้อมกับ factory constructor fromJson

```dart
// lib/models/todo.dart
class Todo {
  final int userId;
  final int id;
  final String title;
  final bool completed;

  const Todo({
    required this.userId,
    required this.id,
    required this.title,
    required this.completed,
  });

  // Factory constructor สำหรับสร้าง Todo instance จาก Map (JSON object)
  factory Todo.fromJson(Map<String, dynamic> json) {
    return Todo(
```

```
      userId: json['userId'] as int,
      id: json['id'] as int,
      title: json['title'] as String,
      completed: json['completed'] as bool,
    );
  }
}
```

ขั้นตอนที่ 3: การสร้างฟังก์ชันสำหรับดึงข้อมูล (Fetching Function)

เราจะสร้างฟังก์ชัน async ที่รับผิดชอบการดึงและแปลงข้อมูล

1. **คำแนะนำ:** สร้างไฟล์ใหม่ lib/services/api_service.dart (เป็นแนวปฏิบัติที่ดีในการแยกโค้ดส่วนนี้ออกมา)
2. **คำแนะนำ:** เขียนฟังก์ชัน fetchTodos

```
// lib/services/api_service.dart
import 'dart:convert'; // สำหรับ jsonDecode
import 'package:http/http.dart' as http;
import '../models/todo.dart';

// ฟังก์ชันนี้จะคืนค่า Future ของ List<Todo>
Future<List<Todo>> fetchTodos() async {
  final response = await http.get(
    Uri.parse('https://jsonplaceholder.typicode.com/todos'),
    // ใส่ header เฉพาะที่ API ต้องการจริง ๆ ที่นี่คือบอกว่าเราขอคำตอบเป็น JSON
    headers: {'Accept': 'application/json'},
  );

  // ตรวจสอบว่า request สำเร็จหรือไม่ (status code 200)
  if (response.statusCode == 200) {
    // หากสำเร็จ, แปลง body ของ response ซึ่งเป็น JSON string
    // ให้กลายเป็น List<dynamic> (เพราะ JSON array คือ list ของ objects)
    final List<dynamic> data = jsonDecode(response.body);

    // ใช้ .map เพื่อแปลงแต่ละ object ใน List<dynamic>
    // ให้กลายเป็น Todo object โดยใช้ Todo.fromJson
    // แล้วแปลงกลับเป็น List<Todo> อีกครั้ง
    return data.map((json) => Todo.fromJson(json)).toList();
  } else {
    // หากไม่สำเร็จ, โยน Exception เพื่อให้ FutureBuilder จัดการ
    // หัวข้อ 12.6 จะปรับปรุงจุดนี้ให้บอกผู้ใช้ได้ว่าเกิดอะไรขึ้นและควรลองใหม่หรือไม่
    throw Exception('Failed to load todos');
```

```
  }
}
```

ขั้นตอนที่ 4: การเชื่อมต่อ UI กับข้อมูลด้วย FutureBuilder (ฉบับสมบูรณ์)

สุดท้าย เราจะสร้างหน้าจอที่ใช้ FutureBuilder เพื่อแสดงผลลัพธ์จาก fetchTodos

1. **คำแนะนำ:** สร้างไฟล์หน้าจอใหม่ lib/screens/todo_list_screen.dart
2. **คำแนะนำ:** สร้าง StatefulWidget และเรียก fetchTodos ใน initState

```dart
// lib/screens/todo_list_screen.dart
import 'package:flutter/material.dart';
import '../models/todo.dart';
import '../services/api_service.dart';

class TodoListScreen extends StatefulWidget {
  const TodoListScreen({super.key});

  @override
  State<TodoListScreen> createState() => _TodoListScreenState();
}

class _TodoListScreenState extends State<TodoListScreen> {
  late Future<List<Todo>> _futureTodos;

  @override
  void initState() {
    super.initState();
    _futureTodos = fetchTodos();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Todo List from API')),
      body: FutureBuilder<List<Todo>>(
        future: _futureTodos,
        builder: (context, snapshot) {
          if (snapshot.connectionState == ConnectionState.waiting) {
            return const Center(child: CircularProgressIndicator());
          } else if (snapshot.hasError) {
            return Center(child: Text('Error: ${snapshot.error}'));
          } else if (snapshot.hasData) {
            final todos = snapshot.data!;
            // ใช้ ListView.builder เพื่อแสดงรายการอย่างมีประสิทธิภาพ
```

```
            return ListView.builder(
              itemCount: todos.length,
              itemBuilder: (context, index) {
                final todo = todos[index];
                return ListTile(
                  leading: CircleAvatar(
                    child: Text(todo.id.toString()),
                  ),
                  title: Text(todo.title),
                  trailing: Icon(
                    todo.completed ? Icons.check_box :
Icons.check_box_outline_blank,
                    color: todo.completed ? Colors.green : Colors.grey,
                  ),
                );
              },
            );
          } else {
            return const Center(child: Text('No todos found.'));
          }
        },
      ),
    );
  }
}
```

3. **คำแนะนำ:** อย่าลืมตั้งค่า TodoListScreen ให้เป็นหน้าแรกใน main.dart

12.6 เมื่อคำขอล้มเหลว: timeout ข้อผิดพลาด และการลองใหม่

โค้ดในหัวข้อที่แล้วทำงานได้ดีเมื่อทุกอย่างเป็นไปตามแผน คือมีสัญญาณอินเทอร์เน็ต เซิร์ฟเวอร์ตอบเร็ว และตอบกลับมาเป็น JSON ที่ถูกต้อง แต่ในการใช้งานจริงบนโทรศัพท์มือถือ เงื่อนไขทั้งสามข้อนี้ผิดพลาดได้ทุกวัน ผู้ใช้เดินเข้าลิฟต์ ผู้ใช้อยู่ในรถไฟฟ้าใต้ดิน เซิร์ฟเวอร์กำลังปรับปรุงระบบ หรือเครือข่ายขั้นกลางส่งหน้าล็อกอินกลับมาแทนข้อมูล JSON

บทที่ 2.2 บอกเราแล้วว่าต้องใช้ try-catch แต่ยังไม่ได้บอกว่าจะดักอะไรบ้าง และจะแปลงสิ่งที่ดักได้ให้เป็นสิ่งที่ผู้ใช้เข้าใจอย่างไร งานเครือข่ายล้มเหลวได้สามแบบซึ่งต้องปฏิบัติต่างกัน

- **ต่อไม่ติดตั้งแต่ต้น** เช่นไม่มีสัญญาณ ตั้งชื่อโฮสต์ผิด หรือไฟร์วอลล์ปิดกั้น กรณีนี้ Future จบด้วยข้อผิดพลาดจากชั้นระบบเครือข่าย ซึ่งบนมือถือมักเป็น SocketException และเมื่อคอมไพล์ลงเว็บจะเป็น ClientException ของแพ็กเกจ http การลองใหม่มีโอกาสสำเร็จเมื่อสัญญาณกลับมา
- **ต่อติดแต่ช้าเกินกว่าจะรอ** ข้อสำคัญคือแพ็กเกจ http ไม่ได้กำหนด timeout ให้โดยปริยาย ถ้าเราไม่กำหนดเอง วงล้อหมุนอาจหมุนค้างเป็นนาทีจนผู้ใช้ปิดแอปทิ้งไปก่อน วิธีกำหนดคือต่อท้ายคำขอด้วยเมธอด .timeout() ซึ่งจะโยน TimeoutException เมื่อครบเวลาที่ตั้งไว้ [9]
- **ต่อติดแต่คำตอบไม่ใช่อย่างที่คาด** ต้องแยก status code สองกลุ่มออกจากกัน กลุ่ม 4xx เช่น 404 คือคำขอของเราเองไม่ถูกต้อง การกดลองใหม่ด้วยคำขอ

เดิมจะได้ผลเดิมเสมอ จึงไม่ควรแสดงปุ่มลองใหม่ ส่วนกลุ่ม 5xx คือความผิดพลาดที่ฝั่งเซิร์ฟเวอร์ การลองใหม่จึงมีเหตุผล นอกจากนี้ถ้าเนื้อหาที่ได้รับไม่ใช่ JSON คำสั่ง jsonDecode จะโยน FormatException และถ้า JSON ขาดฟิลด์ที่เราคาดไว้ การแปลงชนิดข้อมูลใน fromJson จะล้มเหลวเป็น TypeError สิ่งที่ไม่ควรทำคือส่งข้อผิดพลาดดิบไปแสดงบนหน้าจอด้วยข้อความอย่าง Error: SocketException: Failed host lookup เพราะผู้ใช้อ่านไม่รู้เรื่องและไม่รู้ว่าต้องทำอะไรต่อ วิธีที่เป็นระบบคือประกาศคลาสข้อผิดพลาดของแอปขึ้นมาเองหนึ่งคลาส ให้เก็บทั้งข้อความภาษาคนและข้อมูลว่าการลองใหม่มีประโยชน์หรือไม่ แล้วแปลงข้อผิดพลาดทุกชนิดให้เป็นคลาสนี้ไว้ที่ชั้นบริการเพียงจุดเดียว ฝั่งหน้าจอจึงไม่ต้องรู้จัก SocketException หรือ TimeoutException เลย

ข้อคิดที่ควรติดตัวไปจากหัวข้อนี้คือ ในโลกที่เวลาตอบกลับคาดเดาไม่ได้ การออกแบบสถานะการรอคอยและสถานะผิดพลาดมีความสำคัญไม่น้อยกว่าการออกแบบหน้าจอที่ข้อมูลครบสมบูรณ์ ลองสังเกตแอปพลิเคชันที่มีผู้ใช้หลักล้านคนซึ่งเราใช้อยู่ทุกวันว่าเขาสื่อสารกับเราอย่างไรในวินาทีที่ข้อมูลยังเดินทางมาไม่ถึง แอปเหล่านั้นแทบไม่ปล่อยให้ผู้ใช้เห็นหน้าจอขาวนิ่ง ๆ แต่จะแสดงภาพโครงร่างของเนื้อหาที่กำลังจะมาถึงไว้ก่อน เพื่อให้ผู้ใช้รู้ว่าระบบกำลังทำงานอยู่และกำลังจะได้เห็นอะไร

ในทางปฏิบัติเราสร้างภาพโครงร่างนั้นได้ด้วยวิดเจ็ตพื้นฐานที่มีอยู่แล้ว เช่นวาง Container สีเทาอ่อนที่มีขนาดเท่ากับข้อความและรูปภาพจริงไว้ในตำแหน่งเดียวกัน แล้วแทนที่ด้วยข้อมูลจริงเมื่อ snapshot มีข้อมูล ข้อดีที่ตามมาโดยไม่ต้องทำอะไรเพิ่มคือหน้าจอไม่กระโดด เพราะขนาดของพื้นที่ถูกจองไว้ตั้งแต่ตอนที่ยังไม่มีข้อมูล

```dart
// lib/services/api_service.dart (ฉบับรับมือความล้มเหลว)
import 'dart:async'; // สำหรับ TimeoutException
import 'dart:convert';
import 'package:http/http.dart' as http;
import '../models/todo.dart';

// ข้อผิดพลาดที่แอปเข้าใจเอง พร้อมข้อความที่นำไปแสดงต่อผู้ใช้ได้ทันที
class ApiException implements Exception {
  final String message; // ข้อความภาษาคน ไม่ใช่ข้อความของระบบ
  final bool canRetry;  // ปุ่มลองใหม่ควรมีประโยชน์หรือไม่
  const ApiException(this.message, {this.canRetry = true});

  @override
  String toString() => message;
}

Future<List<Todo>> fetchTodos() async {
  final uri = Uri.parse('https://jsonplaceholder.typicode.com/todos');
  try {
    // http ไม่มี timeout ให้โดยปริยาย ต้องกำหนดเองทุกคำขอ
    final response = await http
        .get(uri, headers: {'Accept': 'application/json'})
        .timeout(const Duration(seconds: 10));

    if (response.statusCode == 200) {
      final data = jsonDecode(response.body) as List<dynamic>;
      return data
          .map((item) => Todo.fromJson(item as Map<String, dynamic>))
          .toList();
    }
    if (response.statusCode >= 500) {
      // ผิดที่ฝั่งเซิร์ฟเวอร์ การลองใหม่มีโอกาสสำเร็จ
```

```
      throw const ApiException('เซิร์ฟเวอร์ขัดข้องชั่วคราว โปรดลองอีกครั้ง');
    }
    // 4xx คือคำขอของเราเองไม่ถูกต้อง ลองใหม่ด้วยคำขอเดิมก็ได้ผลเดิม
    throw ApiException(
      'ไม่พบข้อมูลที่ร้องขอ (รหัส ${response.statusCode})',
      canRetry: false,
    );
  } on TimeoutException {
    throw const ApiException('เซิร์ฟเวอร์ตอบกลับช้าเกินไป โปรดลองอีกครั้ง');
  } on FormatException {
    // คำตอบไม่ใช่ JSON เช่นโดนหน้าล็อกอินของเครือข่ายขั้นกลางคั่นไว้
    throw const ApiException('รูปแบบข้อมูลที่ได้รับไม่ถูกต้อง', canRetry: false);
  } on ApiException {
    rethrow; // โยนต่อ ไม่ให้ตัวดักบรรทัดล่างกลืนข้อความที่เราตั้งใจไว้
  } catch (_) {
    // เหลือกรณีต่อไม่ติด เช่นไม่มีสัญญาณ หรือหาชื่อโฮสต์ไม่พบ
    throw const ApiException('เชื่อมต่ออินเทอร์เน็ตไม่ได้ โปรดตรวจสอบการเชื่อมต่อ');
  }
}
```

เมื่อชั้นบริการรับประกันว่าข้อผิดพลาดทุกชนิดจะออกมาเป็น ApiException ฝั่งหน้าจอก็เหลืองานเพียงสองอย่าง คือแสดงข้อความจากข้อผิดพลาดนั้น และแสดงปุ่มลองใหม่เมื่อค่า canRetry เป็นจริง ส่วนการลองใหม่กับ FutureBuilder มีข้อสำคัญที่บทเรียนส่วนใหญ่ข้ามไป คือ Future หนึ่งตัวให้ผลลัพธ์ได้เพียงครั้งเดียวตลอดอายุของมัน การกดลองใหม่จึงไม่ใช่การสั่งให้ Future ตัวเดิมทำงานซ้ำ แต่เป็นการสร้าง Future ตัวใหม่แล้วมอบให้ FutureBuilder ผ่าน setState

ตัวอย่างต่อไปนี้รวมสามเรื่องเข้าด้วยกัน คือลำดับการตรวจ snapshot ที่ไม่ทำให้รายการกระพริบตอนรีเฟรช ปุ่มลองใหม่ที่ขึ้นตามเงื่อนไข และการลากลงเพื่อรีเฟรชด้วย RefreshIndicator [10] ซึ่งเป็นท่ามาตรฐานที่ผู้ใช้โทรศัพท์คุ้นเคยอยู่แล้ว

```
// lib/screens/todo_list_screen.dart (เฉพาะส่วนที่เปลี่ยนจากหัวข้อ 12.5)
class _TodoListScreenState extends State<TodoListScreen> {
  late Future<List<Todo>> _futureTodos;

  @override
  void initState() {
    super.initState();
    _futureTodos = fetchTodos();
  }

  // สั่งโหลดใหม่ คือสร้าง Future ตัวใหม่ แล้วบอก FutureBuilder ว่าตัวเดิมเลิกใช้แล้ว
  Future<void> _reload() async {
    final future = fetchTodos();
    setState(() => _futureTodos = future);
    try {
      await future; // RefreshIndicator ต้องรู้ว่างานจบแล้วจึงหยุดหมุน
    } catch (_) {
      // ไม่ต้องทำอะไรที่นี่ เพราะ FutureBuilder จะแสดงข้อผิดพลาดให้เอง
```

```dart
  }
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('Todo List from API')),
    body: RefreshIndicator(
      onRefresh: _reload,
      child: FutureBuilder<List<Todo>>(
        future: _futureTodos,
        builder: (context, snapshot) {
          // 1. ตรวจข้อผิดพลาดก่อน แล้วอ่านข้อความจาก ApiException
          if (snapshot.hasError) {
            final error = snapshot.error;
            final message = error is ApiException
                ? error.message
                : 'เกิดข้อผิดพลาดที่ไม่คาดคิด';
            final canRetry = error is! ApiException || error.canRetry;
            return ListView(
              // ต้องเป็นวิดเจ็ตที่เลื่อนได้ ไม่งั้นลากลงเพื่อรีเฟรชไม่ได้
              padding: const EdgeInsets.only(top: 120),
              children: [
                Text(message, textAlign: TextAlign.center),
                const SizedBox(height: 12),
                if (canRetry)
                  Center(
                    child: ElevatedButton(
                      onPressed: _reload,
                      child: const Text('ลองใหม่'),
                    ),
                  ),
              ],
            );
          }
          // 2. ยังไม่มีข้อมูลเลยจึงแสดงวงล้อหมุน ถ้ามีข้อมูลรอบก่อนค้างอยู่
          //    จะข้ามเงื่อนไขนี้ไป รายการเดิมจึงไม่กระพริบขณะรีเฟรช
          if (!snapshot.hasData) {
            return const Center(child: CircularProgressIndicator());
          }
          final todos = snapshot.data!;
          return ListView.builder(
            itemCount: todos.length,
            itemBuilder: (context, index) => ListTile(
              leading: CircleAvatar(child: Text(todos[index].id.toString())),
              title: Text(todos[index].title),
            ),
```

```
            );
          },
        ),
      ),
    );
  }
}
```

12.7 เมื่อการแปลง JSON กินเวลาเกินหนึ่งเฟรม

หัวข้อ 12.2 ระบุว่าเฟรมเวิร์กมีเวลาราว 16 มิลลิวินาทีต่อหนึ่งเฟรม และงานเครือข่ายต้องเป็นอะซิงโครนัสเพราะใช้เวลามากกว่านั้นหลายเท่า แต่มีงานอีกชิ้นหนึ่งในกระบวนการเดียวกันที่มักถูกมองข้าม นั่นคือการแปลงข้อมูล คำสั่ง jsonDecode และการวนสร้างอ็อบเจกต์ด้วย fromJson เป็นการคำนวณแบบซิงโครนัสล้วน ๆ ซึ่งทำงานบน UI isolate ตัวเดียวกับที่วาดหน้าจอ

จุดที่เข้าใจผิดกันบ่อยคือคิดว่าการใส่คำว่า async และ await ทำให้งานหนักย้ายไปทำที่อื่น ความจริงคือ await เพียงบอกว่าให้หยุดรอผลของงานที่ทำอยู่ที่อื่นเท่านั้น ตัว jsonDecode ยังทำงานอยู่บน isolate เดิม และระหว่างที่มันทำงาน เฟรมเวิร์กวาดหน้าจอไม่ได้เลย ถ้าข้อมูลมีสองสามร้อยรายการจะไม่ทันสังเกต แต่ถ้าเป็นข้อมูลหลายพันรายการหรือหลายร้อยกิโลไบต์ ผู้ใช้จะเห็นภาพค้างไปหนึ่งจังหวะก่อนรายการจะปรากฏ ซึ่งเป็นอาการกระตุกที่เกิดตอนโหลดเสร็จ ไม่ใช่ตอนกำลังโหลด

ทางแก้คือย้ายงานแปลงข้อมูลไปทำใน isolate อื่นด้วยฟังก์ชัน compute ที่ Flutter เตรียมไว้ให้สำหรับงานคำนวณแบบยิงครั้งเดียว [11], [12] โดยมีข้อบังคับสองข้อ ข้อแรกฟังก์ชันที่ส่งให้ compute ต้องเป็นฟังก์ชันระดับบนสุดของไฟล์หรือเมธอดแบบ static เพราะ isolate ใหม่เข้าถึงตัวแปรแวดล้อมของอ็อบเจกต์เดิมไม่ได้ ข้อสองค่าที่ส่งเข้าและรับออกต้องเป็นข้อมูลที่ส่งข้าม isolate ได้ ซึ่งข้อความ ตัวเลข List และ Map ที่เก็บค่าพื้นฐานผ่านเกณฑ์นี้ทั้งหมด

```dart
// lib/services/api_service.dart (เฉพาะส่วนที่เกี่ยวกับการแปลงข้อมูล)
import 'package:flutter/foundation.dart'; // สำหรับ compute

// ต้องเป็นฟังก์ชันระดับบนสุด เพราะ isolate ใหม่เข้าถึงตัวแปรของคลาสเดิมไม่ได้
List<Todo> _parseTodos(String responseBody) {
  final data = jsonDecode(responseBody) as List<dynamic>;
  return data
      .map((item) => Todo.fromJson(item as Map<String, dynamic>))
      .toList();
}

Future<List<Todo>> fetchTodos() async {
  final response = await http
      .get(Uri.parse('https://jsonplaceholder.typicode.com/todos'))
      .timeout(const Duration(seconds: 10));

  if (response.statusCode != 200) {
    throw const ApiException('โหลดข้อมูลไม่สำเร็จ');
  }
  // ส่งข้อความ JSON ไปแปลงใน isolate อื่น UI isolate จึงว่างไปวาดเฟรมต่อไปได้
```

```
  return compute(_parseTodos, response.body);
}
```

เกณฑ์การตัดสินใจคือวัดก่อนแล้วจึงย้าย การสร้าง isolate ใหม่มีต้นทุนของตัวเองในระดับไม่กี่มิลลิวินาที ถ้าข้อมูลมีขนาดเล็ก การเรียก compute อาจแพงกว่าการแปลงตรง ๆ เสียอีก แนวทางที่ใช้ได้จริงคือเขียนแบบตรงไปตรงมาก่อน แล้วรันในโหมด profile เพื่อดูว่ามีเฟรมใดใช้เวลาเกินงบประมาณในจังหวะที่ข้อมูลโหลดเสร็จหรือไม่ ถ้าพบจึงย้ายไป compute

ข้อควรระวังอีกข้อคือโค้ดที่ทำงานใน isolate อื่นแตะต้องวิดเจ็ต BuildContext หรือสถานะของแอปไม่ได้เลย หน้าที่ของมันมีเพียงรับข้อมูลเข้า คำนวณ และคืนผลลัพธ์ออกมา

12.8 การนำข้อมูลจากเครือข่ายเข้าสู่ Provider

FutureBuilder เหมาะกับหน้าจอที่ดึงข้อมูลมาแสดงเพียงครั้งเดียว และไม่มีหน้าจออื่นต้องใช้ข้อมูลชุดเดียวกัน แต่เมื่อแอปพลิเคชันซับซ้อนขึ้น ข้อมูลชุดหนึ่งมักถูกใช้จากหลายหน้าจอ ต้องสั่งโหลดซ้ำได้ และอาจถูกแก้ไขจากที่อื่น ในกรณีเช่นนี้ควรย้ายตรรกะการดึงข้อมูลไปไว้ในชั้นจัดการสถานะที่ได้ศึกษาใน บทที่ 10 และ บทที่ 11 นั่นคือ ChangeNotifier ที่ทำงานร่วมกับ Provider

ตัวอย่างที่เห็นภาพชัดที่สุดคือข้อมูลตะกร้าสินค้าของแอปพลิเคชันซื้อขายสินค้า ข้อมูลชุดนี้ไม่ได้อยู่แค่หน้าเดียว มันต้องแสดงจำนวนรายการที่ไอคอนมุมขวาบนของหน้าแรก ต้องแสดงรายการทั้งหมดในหน้าค้นหาสินค้า และต้องแสดงยอดรวมในหน้าชำระเงิน ถ้าแต่ละหน้าจอใช้ FutureBuilder ของตัวเองดึงข้อมูลชุดเดียวกันซ้ำ ผลที่ได้คือเปลืองปริมาณข้อมูลของผู้ใช้โดยไม่จำเป็น และที่แย่กว่านั้นคือข้อมูลระหว่างหน้าจออาจไม่ตรงกัน

เกณฑ์ตัดสินที่ใช้ได้จริงคือดูที่**วงจรชีวิตของข้อมูล** ไม่ใช่ดูที่ความยากง่ายของเครื่องมือ ถ้าข้อมูลมีชีวิตอยู่ยาวนานกว่าหน้าจอใดหน้าจอหนึ่ง ต้องถูกใช้ร่วมกันข้ามหน้าจอ หรือต้องสั่งโหลดซ้ำได้จากที่อื่น ตรรกะการดึงข้อมูลควรย้ายไปอยู่ใน ChangeNotifier ซึ่งทำหน้าที่เป็นศูนย์บัญชาการของข้อมูลชุดนั้น ส่วนถ้าข้อมูลเกิดและตายไปพร้อมกับหน้าจอเดียว FutureBuilder ก็เพียงพอและอ่านง่ายกว่า

แนวคิดคือให้ ChangeNotifier เก็บทั้งข้อมูลและสถานะการทำงานไว้ด้วยกัน เมื่อเริ่มดึงข้อมูลให้ตั้งค่าสถานะกำลังโหลดแล้วเรียก notifyListeners() หนึ่งครั้ง เมื่อได้ผลลัพธ์หรือเกิดข้อผิดพลาดจึงอัปเดตค่าและเรียก notifyListeners() อีกครั้ง ส่วนฝั่ง UI ใช้ Consumer ตามที่เรียนในบทที่ 11 เพื่อวาดใหม่เฉพาะส่วนที่จำเป็น

ก่อนใช้โค้ดในหัวข้อนี้ ต้องเพิ่มแพ็กเกจ provider ลงในไฟล์ pubspec.yaml เพิ่มเติมจาก http ตามที่เคยทำในบทที่ 10 และครอบวิดเจ็ตรากของแอปด้วย ChangeNotifierProvider ใน main.dart เช่นเดียวกับที่ปฏิบัติไว้ในบทที่ 11

```
// lib/providers/todo_provider.dart
import 'package:flutter/material.dart';
import '../models/todo.dart';
import '../services/api_service.dart';

class TodoProvider extends ChangeNotifier {
  List<Todo> _todos = [];
  bool _isLoading = false;
  String? _errorMessage;

  // คืนเป็นสำเนาที่แก้ไขไม่ได้ เพื่อบังคับให้การเปลี่ยนแปลงข้อมูลทุกครั้ง
  // ต้องผ่านเมธอดของคลาสนี้ ซึ่งจะเรียก notifyListeners() ให้เสมอ
  // (หากคืน _todos ตรง ๆ หน้าจออาจแก้ไขรายการได้โดยที่ UI ไม่ถูกแจ้งให้วาดใหม่)
```

```
  List<Todo> get todos => List.unmodifiable(_todos);
  bool get isLoading => _isLoading;
  String? get errorMessage => _errorMessage;

  Future<void> loadTodos() async {
    _isLoading = true;
    _errorMessage = null;
    notifyListeners(); // แจ้ง UI ให้แสดงสถานะกำลังโหลด

    try {
      _todos = await fetchTodos(); // ใช้ฟังก์ชันเดิมจากหัวข้อ 12.6
    } on ApiException catch (e) {
      // ชั้นบริการแปลงข้อผิดพลาดทุกชนิดมาเป็นข้อความภาษาคนให้แล้ว
      _errorMessage = e.message;
      // ไม่ล้าง _todos ทิ้ง เพื่อให้ผู้ใช้ยังเห็นข้อมูลรอบก่อนระหว่างที่โหลดซ้ำไม่สำเร็จ
    } catch (e) {
      _errorMessage = 'ไม่สามารถโหลดข้อมูลได้';
    } finally {
      _isLoading = false;
      notifyListeners(); // แจ้ง UI อีกครั้งเมื่อได้ผลลัพธ์แล้ว
    }
  }
}
```

ฝั่งหน้าจอต้องสั่ง loadTodos() เพียงครั้งเดียวเมื่อเปิดหน้าจอ แต่มีหลุมพรางอยู่ตรงนี้ ถ้าเขียน context.read<TodoProvider>().loadTodos(); ไว้ใน initState ตรง ๆ แอปจะหยุดทำงานทันทีด้วยข้อความว่า setState() or markNeedsBuild() called during build. เหตุผลคือบรรทัดที่สามของ loadTodos() เรียก notifyListeners() แบบทันที ขณะที่จังหวะของ initState ยังอยู่ในเฟสสร้างวิดเจ็ตของเฟรมแรก การแจ้งให้ผู้รับฟังสร้างตัวเองใหม่ในจังหวะที่เฟรมยังสร้างไม่เสร็จจึงเป็นสิ่งที่เฟรมเวิร์กไม่อนุญาต วิธีแก้คือเลื่อนการเรียกออกไปให้พ้นเฟรมแรกด้วย addPostFrameCallback

```
// lib/screens/todo_provider_screen.dart (ส่วนวงจรชีวิตของหน้าจอ)
class _TodoProviderScreenState extends State<TodoProviderScreen> {
  @override
  void initState() {
    super.initState();
    // เลื่อนไปเรียกหลังเฟรมแรกวาดเสร็จ เพราะ loadTodos() เรียก notifyListeners() ทันที
    WidgetsBinding.instance.addPostFrameCallback((_) {
      if (!mounted) return; // ผู้ใช้อาจกดย้อนกลับออกไปก่อนเฟรมแรกจบ
      context.read<TodoProvider>().loadTodos();
    });
  }
```

อีกทางเลือกหนึ่งคือออกแบบให้ loadTodos() ไม่เรียก notifyListeners() ในรอบแรกเมื่อยังไม่มีผู้รับฟัง แต่วิธี addPostFrameCallback อ่านง่ายกว่าและใช้ได้กับ provider ทุกตัวโดยไม่ต้องแก้ตรรกะภายใน ข้อสำคัญคือต้องตรวจ mounted ในฟังก์ชันที่เลื่อนไปเรียกด้วย เพราะผู้ใช้อาจกดย้อนกลับออกจากหน้าจอไปแล้วตามเหตุผลเดียวกับที่กล่าวไว้ในหัวข้อ 12.2

จากนั้นใช้ Consumer<TodoProvider> เลือกแสดงผลตามสถานะปัจจุบัน โดยเพิ่มปุ่มลองใหม่ไว้ในกรณีผิดพลาด และคงรายการเดิมไว้เมื่อกำลังโหลดซ้ำทับข้อมูลที่มีอยู่แล้ว ซึ่งเป็นหลักเดียวกับที่ใช้กับ FutureBuilder ในหัวข้อ 12.6

```dart
// lib/screens/todo_provider_screen.dart (เฉพาะส่วน build)
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('Todo List with Provider')),
    body: Consumer<TodoProvider>(
      builder: (context, provider, child) {
        // กำลังโหลดครั้งแรก คือยังไม่มีข้อมูลอะไรให้แสดงเลย
        if (provider.isLoading && provider.todos.isEmpty) {
          return const Center(child: CircularProgressIndicator());
        }
        // ผิดพลาดและไม่มีข้อมูลเดิมค้างอยู่ จึงแสดงข้อความพร้อมปุ่มลองใหม่
        if (provider.errorMessage != null && provider.todos.isEmpty) {
          return Center(
            child: Column(
              mainAxisAlignment: MainAxisAlignment.center,
              children: [
                Text(provider.errorMessage!, textAlign: TextAlign.center),
                const SizedBox(height: 12),
                ElevatedButton(
                  onPressed: () => context.read<TodoProvider>().loadTodos(),
                  child: const Text('ลองใหม่'),
                ),
              ],
            ),
          );
        }
        return ListView.builder(
          itemCount: provider.todos.length,
          itemBuilder: (context, index) {
            return ListTile(title: Text(provider.todos[index].title));
          },
        );
      },
    ),
    floatingActionButton: FloatingActionButton(
      // สั่งโหลดซ้ำได้โดยไม่ต้องสร้างหน้าจอใหม่ ซึ่งทำได้ยากกว่าเมื่อใช้ FutureBuilder
      onPressed: () => context.read<TodoProvider>().loadTodos(),
      child: const Icon(Icons.refresh),
    ),
  );
}
```

จุดสำคัญที่ควรจดจำคือ งานอะซิงโครนัสทุกชนิดมีสามสถานะเสมอ ได้แก่ กำลังโหลด สำเร็จ และผิดพลาด ไม่ว่าเราจะจัดการด้วย FutureBuilder หรือด้วย ChangeNotifier ก็ตาม โครงสร้างสามสถานะนี้จะถูกนำไปใช้ซ้ำใน บทที่ 13 โดยเปลี่ยนแหล่งข้อมูลจาก API บนเครือข่ายไปเป็นฐานข้อมูล SQLite บนตัวเครื่อง ซึ่งทุกคำสั่งของแพ็กเกจ sqflite ล้วนคืนค่าเป็น Future เช่นเดียวกัน

โครงสร้างนี้คือคำตอบของคำถามท้ายบทที่ 11 อย่างครบถ้วน คำถามข้อแรกว่าระหว่างที่ข้อมูลยังมาไม่ถึง provider ควรบอกหน้าจอว่าอย่างไร คำตอบคือบอกด้วยค่า isLoading ไม่ใช่บอกด้วยรายการว่าง เพราะรายการว่างมีความหมายอยู่แล้วว่าดึงข้อมูลสำเร็จแต่ไม่มีรายการ ส่วนคำถามข้อที่สองว่าใครควรจำข้อผิดพลาดไว้เพื่อแสดงปุ่มลองใหม่ คำตอบคือ provider เป็นผู้จำ เพราะข้อผิดพลาดเป็นสถานะของข้อมูลชุดนั้น ไม่ใช่สถานะของหน้าจอใดหน้าจอหนึ่ง เมื่อผู้ใช้สลับไปหน้าอื่นแล้วกลับมา สถานะเดิมจึงยังอยู่ครบและไม่ยิงคำขอซ้ำโดยไม่จำเป็น

ควรเลือกใช้วิธีใด

- **เลือก FutureBuilder เมื่อ:** ข้อมูลถูกใช้เฉพาะในหน้าจอเดียว ดึงครั้งเดียวตอนเปิดหน้าจอ และไม่จำเป็นต้องแก้ไขหรือสั่งโหลดซ้ำจากที่อื่น
- **เลือก Provider เมื่อ:** ข้อมูลถูกใช้ร่วมกันหลายหน้าจอ ต้องสั่งโหลดซ้ำหรือแก้ไขได้ หรือต้องคงอยู่ขณะสลับไปมาระหว่างหน้าจอ ซึ่งเป็นรูปแบบที่ใช้ในปฏิบัติการของบทที่ 13

12.9 สรุปบทเรียนและสิ่งที่จะเรียนรู้ต่อไป

บทนี้ต่อยอดพื้นฐานอะซิงโครนัสจากบทที่ 2.2 มาสู่การใช้งานจริงบน Flutter จนได้แอปพลิเคชันที่ดึงและแสดงข้อมูลจากอินเทอร์เน็ตได้สำเร็จ ประเด็นสำคัญของบทนี้คือ

- **ข้อจำกัดของ UI isolate:** โค้ดของแอปและการวาดหน้าจอใช้ isolate เดียวกัน และมีเวลาราว 16 มิลลิวินาทีต่อเฟรม งานเครือข่ายจึงต้องเป็นอะซิงโครนัสเสมอ
- **รูปแบบการใช้ async ในวิดเจ็ต:** เริ่มงานอะซิงโครนัสใน initState ไม่ใช่ใน build, ตรวจสอบ mounted ก่อนเรียก setState หลัง await และระวังการใช้ BuildContext ข้าม await
- **FutureBuilder:** เป็นวิดเจ็ตที่จำเป็นอย่างยิ่งสำหรับการสร้าง UI ที่ขึ้นอยู่กับผลลัพธ์ของ Future โดยช่วยจัดการสถานะการโหลด, ข้อผิดพลาด, และข้อมูลให้โดยอัตโนมัติ
- **กระบวนการดึงข้อมูล:** ตั้งแต่การเพิ่ม http package, การสร้าง Model Class จาก JSON, การเขียนฟังก์ชันดึงข้อมูล, ไปจนถึงการแสดงผลใน ListView
- **การเชื่อมกับ Provider:** การเก็บสถานะกำลังโหลด สำเร็จ และผิดพลาดไว้ใน ChangeNotifier ทำให้ข้อมูลจากเครือข่ายถูกใช้ร่วมกันได้หลายหน้าจอและสั่งโหลดซ้ำได้
- การรับมือความล้มเหลว: กำหนด timeout ให้ทุกคำขอเพราะแพ็กเกจ http ไม่ได้ให้มาโดยปริยาย แยก 4xx ที่ลองใหม่ไม่ช่วยออกจาก 5xx ที่ลองใหม่มีเหตุผล และแปลงข้อผิดพลาดทุกชนิดเป็นคลาสของแอปเองที่พาข้อความภาษาคนไปถึงหน้าจอ
- การลองใหม่: Future หนึ่งตัวให้ผลเพียงครั้งเดียว การลองใหม่จึงหมายถึงการสร้าง Future ตัวใหม่แล้วส่งให้ FutureBuilder ผ่าน setState หรือเรียกเมธอดของ provider ซ้ำ
- การแปลง JSON ก้อนใหญ่: jsonDecode ทำงานบน UI isolate การใส่ async ไม่ได้ย้ายงานไปที่อื่น ถ้าวัดแล้วพบว่าเกินงบประมาณต่อเฟรมจึงย้ายไปทำใน isolate อื่นด้วย compute
- สิทธิ์เข้าถึงเครือข่าย: Android ต้องประกาศสิทธิ์ INTERNET ในไฟล์ manifest ของ main และ macOS ต้องเปิด entitlement ของ network client ไม่เช่นนั้นแอปฉบับ release จะเชื่อมต่อไม่ได้ทั้งที่โค้ดถูกต้อง

ตอนนี้คุณมีทักษะที่จำเป็นในการทำให้แอปของคุณเชื่อมต่อกับโลกภายนอกและแสดงข้อมูลแบบไดนามิกได้แล้ว

บทต่อไป:

ข้อมูลที่เราดึงมาในบทนี้จะหายไปทุกครั้งที่ปิดแอป ใน บทที่ 13: การคงอยู่ของข้อมูลด้วย SQLite เราจะเรียนรู้วิธีบันทึกข้อมูลลงฐานข้อมูลบนอุปกรณ์โดยตรง เพื่อให้แอปทำงานแบบออฟไลน์และจดจำข้อมูลของผู้ใช้ได้ คำสั่งของแพ็กเกจ sqflite ทุกคำสั่งคืนค่าเป็น Future ดังนั้นทักษะ async/await และรูปแบบสามสถานะที่ฝึกในบทนี้จะถูกนำไปใช้ต่อทันที โดยเฉพาะในปฏิบัติการของบทที่ 13 ที่ใช้ Provider เป็นศูนย์กลางในการติดต่อฐานข้อมูล

12.10 คำถามชวนคิดท้ายบท

ลองนึกถึงผู้ใช้คนหนึ่งที่เปิดแอปของเราดูรายการทั้งหมดตอนเช้าที่บ้านซึ่งสัญญาณดี แล้วตอนสายเดินเข้าลิฟต์และเปิดแอปขึ้นมาอีกครั้งเพื่อดูรายการเดิม สิ่งที่เขาเห็นคือวงล้อหมุนสักครู่ ตามด้วยข้อความว่าเชื่อมต่ออินเทอร์เน็ตไม่ได้และปุ่มลองใหม่ที่กดแล้วก็ยังไม่ได้ผล ทั้งที่ข้อมูลชุดเดียวกันนั้นเพิ่งเดินทางมาถึงเครื่องของเขาเมื่อไม่กี่ชั่วโมงก่อน

เหตุที่เป็นเช่นนั้นเพราะทุกอย่างที่เราสร้างในบทนี้ถือข้อมูลไว้ในหน่วยความจำของแอปเท่านั้น เมื่อแอปถูกปิด ข้อมูลหายไปทั้งหมด และแหล่งความจริงเพียงแหล่งเดียวที่เรายอมรับคือเซิร์ฟเวอร์ปลายทาง ถ้าไปไม่ถึงก็ไม่มีอะไรจะแสดง

คำถามคือ ถ้าเราต้องการให้ผู้ใช้เปิดแอปมาเห็นรายการที่โหลดสำเร็จครั้งล่าสุดได้ทันทีแม้ไม่มีสัญญาณ ข้อมูลชุดนั้นควรถูกเก็บไว้ที่ไหนและใครควรเป็นผู้เก็บ ตรรกะการอ่านข้อมูลจากที่เก็บนั้นควรอยู่ในชั้นเดียวกับ fetchTodos() หรือควรเป็นอีกชั้นหนึ่งที่ provider เรียกใช้

และคำถามที่ยากกว่านั้นคือ เมื่อมีข้อมูลสองชุดคือชุดที่เก็บไว้ในเครื่องกับชุดที่อยู่บนเซิร์ฟเวอร์ ชุดใดคือความจริง ถ้าผู้ใช้เห็นข้อมูลเก่าอยู่แล้วเบื้องหลังกำลังดึงชุดใหม่มาทับ เราจะบอกผู้ใช้อย่างไรว่าสิ่งที่เห็นเป็นข้อมูลเมื่อเช้า สามสถานะที่เราใช้ทั้งบทนี้ คือกำลังโหลด สำเร็จ และผิดพลาด ยังเพียงพอหรือไม่ หรือต้องมีสถานะที่สี่ว่ามีข้อมูลเก่าอยู่และกำลังรีเฟรช

ลองตอบคำถามเหล่านี้ด้วยเครื่องมือที่มีอยู่ในมือแล้วก่อน จากนั้นนำคำตอบของตัวเองไปเทียบกับบทที่ 13: การคงอยู่ของข้อมูลด้วย SQLite ซึ่งจะเพิ่มที่เก็บข้อมูลบนตัวเครื่องเข้ามาเป็นอีกแหล่งหนึ่ง โดยที่คำสั่งทุกคำสั่งของแพ็กเกจ sqflite คืนค่าเป็น Future เช่นเดียวกับงานเครือข่ายในบทนี้ทั้งหมด

เอกสารอ้างอิง (References)

[1] Dart Team, "Asynchronous programming," *Dart Documentation*. Accessed: Sep. 24, 2026. [Online]. Available: https://dart.dev/language/async

[2] Flutter Team, "Concurrency and isolates," *Flutter Documentation*. Accessed: Sep. 24, 2026. [Online]. Available: https://docs.flutter.dev/perf/isolates

[3] Flutter Team, "Performance best practices," *Flutter Documentation*. Accessed: Sep. 24, 2026. [Online]. Available: https://docs.flutter.dev/perf/best-practices

[4] Flutter Team, "FutureBuilder class," *Flutter API Docs*. Accessed: Sep. 24, 2026. [Online]. Available: https://api.flutter.dev/flutter/widgets/FutureBuilder-class.html

[5] Flutter Team, "Fetch data from the internet," *Flutter Documentation*. Accessed: Sep. 24, 2026. [Online]. Available: https://docs.flutter.dev/cookbook/networking/fetch-data

[6] Dart Team, "http | Dart package," *pub.dev*. Accessed: Sep. 24, 2026. [Online]. Available: https://pub.dev/packages/http

[7] Flutter Team, "Networking," *Flutter Documentation*. Accessed: Sep. 24, 2026. [Online]. Available: https://docs.flutter.dev/data-and-backend/networking

[8] Flutter Team, "JSON and serialization," *Flutter Documentation*. Accessed: Sep. 24, 2026. [Online]. Available: https://docs.flutter.dev/data-and-backend/serialization/json

[9] Dart Team, "Future.timeout method," *Dart API Docs*. Accessed: Sep. 24, 2026. [Online]. Available: https://api.dart.dev/stable/dart-async/Future/timeout.html

[10] Flutter Team, "RefreshIndicator class," *Flutter API Docs*. Accessed: Sep. 24, 2026. [Online]. Available: https://api.flutter.dev/flutter/material/RefreshIndicator-class.html

[11] Flutter Team, "Parse JSON in the background," *Flutter Documentation*. Accessed: Sep. 24, 2026. [Online]. Available: https://docs.flutter.dev/cookbook/networking/background-parsing

[12] Flutter Team, "compute function," *Flutter API Docs*. Accessed: Sep. 24, 2026. [Online]. Available: https://api.flutter.dev/flutter/foundation/compute.html

กิจกรรมปฏิบัติการบทที่ 12: สร้างแอปแสดงรายการโพสต์จาก API

วัตถุประสงค์: เพื่อนำความรู้ทั้งหมดในบทนี้มาใช้สร้างแอปพลิเคชันที่สมบูรณ์ ซึ่งดึงรายการโพสต์จาก JSONPlaceholder API และแสดงผลออกมาในรูปแบบที่สวยงาม

เวลาโดยประมาณ: 2 ชั่วโมง

ขั้นตอนที่ 1: ตั้งค่าโปรเจกต์

1. **คำแนะนำ:** สร้างโปรเจกต์ Flutter ใหม่ชื่อ api_post_viewer
2. **คำแนะนำ:** เปิด pubspec.yaml และเพิ่ม http package ใน dependencies แล้วรัน flutter pub get

```
dependencies:
  flutter:
    sdk: flutter
  http: ^1.5.0
```

ขั้นตอนที่ 2: สร้าง Model และ Service

1. **คำแนะนำ:** สร้างไฟล์ lib/post_model.dart สำหรับข้อมูล Post. ข้อมูลจาก https://jsonplaceholder.typicode.com/posts มีลักษณะดังนี้: {"userId": 1, "id": 1, "title": "...", "body": "..."}

```
// lib/post_model.dart
class Post {
  final int id;
  final String title;
  final String body;

  const Post({required this.id, required this.title, required this.body});
```

```
  factory Post.fromJson(Map<String, dynamic> json) {
    return Post(
      id: json['id'] as int,
      title: json['title'] as String,
      body: json['body'] as String,
    );
  }
}
```

2. **คำแนะนำ:** สร้างไฟล์ lib/api_service.dart และเพิ่มฟังก์ชัน fetchPosts

```
// lib/api_service.dart
import 'dart:convert';
import 'package:http/http.dart' as http;
import 'package:api_post_viewer/post_model.dart';

Future<List<Post>> fetchPosts() async {
  final response = await http.get(
    Uri.parse('https://jsonplaceholder.typicode.com/posts'),
    headers: {'Accept': 'application/json'},
  ).timeout(const Duration(seconds: 10));

  if (response.statusCode == 200) {
    final List<dynamic> data = jsonDecode(response.body);
    return data.map((json) => Post.fromJson(json)).toList();
  } else {
    throw Exception('Failed to load posts');
  }
}
```

ข้อควรระวังเรื่องเส้นทางของ import: ชื่อหลัง package: ในคำสั่ง import ต้องเป็นชื่อโปรเจกต์ตามที่ประกาศไว้ในไฟล์ pubspec.yaml และเส้นทางที่ตามมาต้องเป็นเส้นทางจริงภายในโฟลเดอร์ lib ในปฏิบัติการนี้ไฟล์ต้นแบบข้อมูลอยู่ที่ lib/post_model.dart ดังนั้นคำสั่งที่ถูกต้องคือ import 'package:api_post_viewer/post_model.dart'; หากตั้งชื่อโปรเจกต์เป็นชื่ออื่นต้องเปลี่ยนตาม หรือจะใช้เส้นทางแบบสัมพัทธ์เป็น import 'post_model.dart'; ก็ได้ผลเหมือนกันเพราะทั้งสองไฟล์อยู่ในโฟลเดอร์เดียวกัน

ขั้นตอนที่ 3: สร้าง UI และเชื่อมต่อข้อมูล

1. **คำแนะนำ:** แทนที่โค้ดทั้งหมดใน lib/main.dart ด้วยโค้ดด้านล่างนี้ โค้ดนี้ประกอบด้วย StatefulWidget (PostListScreen) ที่เรียก fetchPosts ใน initState และใช้ FutureBuilder เพื่อแสดงผลลัพธ์

```dart
// lib/main.dart
import 'package:flutter/material.dart';
import 'api_service.dart';
import 'post_model.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'API Post Viewer',
      theme: ThemeData(
        primarySwatch: Colors.teal,
        visualDensity: VisualDensity.adaptivePlatformDensity,
      ),
      home: const PostListScreen(),
    );
  }
}

class PostListScreen extends StatefulWidget {
  const PostListScreen({super.key});

  @override
  State<PostListScreen> createState() => _PostListScreenState();
}

class _PostListScreenState extends State<PostListScreen> {
  late Future<List<Post>> _futurePosts;

  @override
  void initState() {
    super.initState();
    _futurePosts = fetchPosts();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Posts from API'),
      ),
```

```
    body: FutureBuilder<List<Post>>(
      future: _futurePosts,
      builder: (context, snapshot) {
        if (snapshot.connectionState == ConnectionState.waiting) {
          return const Center(child: CircularProgressIndicator());
        } else if (snapshot.hasError) {
          return Center(child: Text("Error: ${snapshot.error}"));
        } else if (snapshot.hasData) {
          return ListView.builder(
            itemCount: snapshot.data!.length,
            itemBuilder: (context, index) {
              Post post = snapshot.data![index];
              return Card(
                margin: const EdgeInsets.symmetric(vertical: 8, horizontal: 16),
                elevation: 4,
                child: Padding(
                  padding: const EdgeInsets.all(16.0),
                  child: Column(
                    crossAxisAlignment: CrossAxisAlignment.start,
                    children: [
                      Text(
                        '${post.id}. ${post.title}',
                        style: const TextStyle(
                          fontSize: 18,
                          fontWeight: FontWeight.bold,
                        ),
                      ),
                      const SizedBox(height: 8),
                      Text(
                        post.body,
                        style: TextStyle(
                          fontSize: 14,
                          color: Colors.grey[700],
                        ),
                      ),
                    ],
                  ),
                ),
              );
            },
          );
        }
        return const Center(child: Text("No posts found."));
      },
    ),
  );
}
```

```
}
```

ขั้นตอนที่ 4: รันแอปพลิเคชัน

1. **คำแนะนำ:** ตรวจสอบให้แน่ใจว่าอุปกรณ์หรือ Emulator ของคุณเชื่อมต่ออินเทอร์เน็ต
2. **คำแนะนำ:** รันแอปพลิเคชัน
3. **ผลลัพธ์ที่คาดหวัง:**
    - คุณจะเห็นหน้าจอ AppBar พร้อมกับวงล้อหมุนตรงกลางชั่วครู่
    - จากนั้นหน้าจอจะแสดงรายการโพสต์ที่เลื่อนได้ โดยแต่ละโพสต์จะแสดงอยู่ใน Card ที่มี title เป็นตัวหนาและ body เป็นข้อความปกติ

ยินดีด้วย! คุณได้สร้างแอปพลิเคชัน Flutter ที่สมบูรณ์ซึ่งสามารถดึง, แปลง, และแสดงข้อมูลจาก API ภายนอกได้อย่างสวยงามและมีประสิทธิภาพแล้ว

ขั้นตอนที่ 5: ความท้าทายเพิ่มเติม

แอปที่ได้ยังรับมือกับความล้มเหลวเพียงระดับพื้นฐาน ให้ทดลองขยายต่อสามข้อต่อไปนี้ โดยข้อแรกและข้อที่สองใช้เนื้อหาจากหัวข้อ 12.6 และข้อที่สามใช้เนื้อหาจากหัวข้อ 12.8

ข้อที่ 1 ปิดโหมดเครื่องบินไว้แล้วรันแอป สังเกตข้อความที่ปรากฏบนหน้าจอ จากนั้นแก้ไขให้แสดงข้อความภาษาไทยที่ผู้ใช้เข้าใจได้ พร้อมปุ่มลองใหม่ที่กดแล้วสร้าง Future ตัวใหม่จริง

ข้อที่ 2 เพิ่ม RefreshIndicator ให้ผู้ใช้ลากลงเพื่อโหลดรายการใหม่ได้ แล้วตรวจว่ารายการเดิมไม่กระพริบหายไปขณะรีเฟรช

ข้อที่ 3 ย้ายตรรกะการดึงข้อมูลจากคลาส State ไปไว้ใน ChangeNotifier ที่ชื่อ PostProvider แล้วเปลี่ยนหน้าจอให้ใช้ Consumer<PostProvider> เทียบผลลัพธ์ทั้งสองแบบว่าต่างกันอย่างไรเมื่อต้องเพิ่มหน้าจอที่สองซึ่งใช้ข้อมูลชุดเดียวกัน